Smart Agricultural Technology 12 (2025) 101156

Contents lists available at ScienceDirect

Smart Agricultural Technology

journal homepage: www.journals.elsevier.com/smart-agricultural-technology

# Fields2Benchmark: An open-source benchmark for coverage path planning methods in agriculture

Gonzalo Mier [a,*], Ana María Casado Faulí [b], João Valente [c], Sytze de Bruin [a]

[a] Laboratory of Geo-Information Science and Remote Sensing, Wageningen University and Research, Wageningen, the Netherlands
[b] Dept. of Mechanics, Royal Military Academy, Brussels, Belgium
[c] Centre for Automation and Robotics (CAR), Spanish National Research Council (CSIC), Madrid, Spain

ARTICLE INFO

Dataset link: https://doi.org/10.5281/zenodo.14524735

*MSC:*
0000
1111

*Keywords:*
Coverage path planning
Route optimization
Benchmark
Field robotics

ABSTRACT

The agricultural coverage path planning problem focuses on optimizing coverage paths for agricultural operations. Despite its importance, existing agricultural coverage path planning solutions are highly application-specific, limiting their generalizability and reproducibility. Fields2Cover offers a modular open source software library for coverage path planning, yet it lacks built-in means to benchmark its algorithms and to configure the workflow to specific tasks. It also fails on non-convex fields and operations that require reload trips. This paper introduces Fields2Benchmark, an open-source, modular benchmark designed to standardize the evaluation of agricultural coverage path planning algorithms. Fields2Benchmark includes a dataset with 350 real-world fields, featuring non-convex fields and in-field obstacles. The benchmark decomposes the problem into five units, i.e., field decomposition, headland generation, swath generation, route planning, and path planning— allowing researchers to evaluate and compare algorithms modularly. Each module supports interchangeable algorithms and objective functions, enabling customization for diverse use cases. Fields2Benchmark extends the existing Fields2Cover library by allowing task-specific choices, adding a module that splits a field into simpler parts, introducing new algorithms to shorten routes and handle refill needs, and adding tools to compare algorithms in isolation. Outputs are recorded as structured numerical data and visual representations to facilitate detailed analysis. The capabilities of the benchmark were validated on three use cases concerning field arrangement and route and path planning with and without capacity constraints. Results demonstrate its ability to handle complex field geometries, compare algorithms effectively, and evaluate computational performance. Fields2Benchmark is computationally efficient, with planning times suitable for real-time applications. It is supported by publicly available datasets and code. By standardizing agricultural coverage path planning evaluation, Fields2Benchmark aims to improve the reproducibility in this field, accelerating the research in agricultural robotics and field operations. For industrial robots, this benchmark cuts deployment costs, and shortens time to market, by simplifying the development process.

## 1. Introduction

The Coverage Path Planning problem for agricultural environments (ACPP) aims to find an optimal path that completely covers a field. Coverage is crucial for tasks such as seeding, mowing, and crop protection, where efficiency and completeness determine crop yield and costs. Solving the ACPP problem typically relies on customized development, leading to the inability to reproduce cross-study results while also compromising generalizability [1]. Despite the need for comparability and reproducibility between different methods, there is no standardized benchmark for ACPP [2]. This lack of benchmarking makes it difficult to compare the performance of different ACPP approaches [3]. This paper fills this gap through open-source code and public datasets.

A standardized benchmark requires open-source code for execution and a public dataset for input [4]. To our knowledge, there is no publicly available code to run an ACPP benchmark. Recently, a benchmark for informative path planning (IPP) [5] was released. The IPP problem searches for the best observable positions to survey an area. Despite their

* Corresponding author.
*E-mail address:* gonzalo.miermunoz@wur.nl (G. Mier).

https://doi.org/10.1016/j.atech.2025.101156
Received 9 April 2025; Received in revised form 28 June 2025; Accepted 1 July 2025
Available online 9 July 2025
2772-3755/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

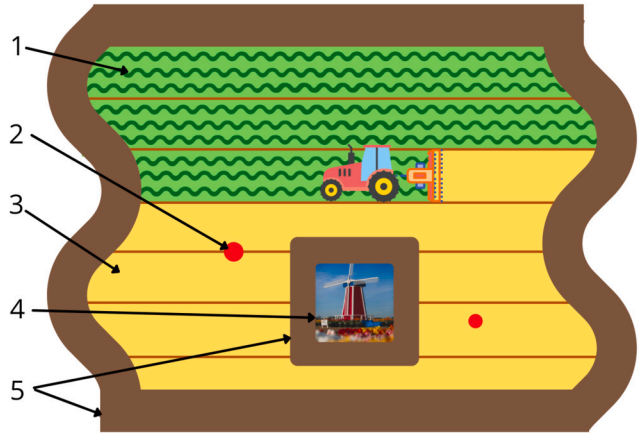



**Fig. 1.** Scheme of a field being harvested. 1- Uncovered swaths. 2- Small obstacles (red dots). 3- Already covered swath. 4- Large obstacle. 5- Headlands.

similarities, the ACPP usually aims to avoid overlap, which is a restriction not imposed to IPP. The only open-source software library available for ACPP is Fields2Cover [6], but its initial release lacked tools for solving non-convex field geometries and did not consider a robot with capacity constraints. Additionally, Fields2Cover did not include benchmark functionalities or provide a public data set for comparative analysis. Other open-source projects [7–9] also focus on the coverage path planning problem, although not for ground vehicles for agriculture. These projects usually provide a single algorithm, limiting their benchmarking capabilities. Today's published datasets fail to capture the complexities of many agricultural fields. For instance, Khosravani [10] provides a single convex field to compare route planners. Nilsson's dataset [11] offers 54 manually-created geometries without in-field obstacles. Recently, Pour [12] introduced a dataset of 30 French fields, but these also lack obstacles, limiting their utility for evaluating ACPP algorithms on more complex problems.

To address these gaps, we developed Fields2Benchmark, an open-source benchmark for ACPP. Fields2Benchmark standardizes ACPP evaluation by dividing the planning process into five separate, replaceable modules: Field Decomposition, Headland Generation, Swath Generation, Route Planning, and Path Planning. This modular framework ensures reproducible comparisons and accelerates algorithm innovation in agricultural robotics. Furthermore, the benchmark is integrated in Fields2Cover[1] to encourage community adoption.

## 2. Background agricultural coverage path planning

The aim of ACPP is to generate a path to cover an agricultural field. Those fields typically consist of headlands and the inner field. Headlands provide space for machinery to turn [13], and they are located along the external boundaries and around large in-field obstacles (Fig. 1, arrow 5). The inner field is covered using swaths [14], which are parallel strips of land that must be covered (Fig. 1, arrows 1 & 3). Similarly to swaths in the inner field, headlands may also have swaths, which are called headland tracks. The goal of ACPP is to plan an efficient route –the sequence in which swaths are traversed– and then connect these swaths into a continuous path that the agricultural vehicle, or robot, can follow [15]. When the robot has a finite operational capacity, for instance, in pesticide application or seeding, reload points become necessary to manage its resource limits [16].

The ACPP problem is commonly divided into distinct steps [17]: Field Decomposition, Headland Generation, Swath Generation, Route Planning, and Path Planning. Field Decomposition divides fields into sub-cells, which is commonly achieved by boustrophedon cellular decomposition, involving two stages: first splitting fields into smaller cells, then merging these cells into larger units. The splitting usually follows a predefined coverage direction [18] or extends edges from the field boundary [19]. Optimization algorithms merge the resulting sub-cells to reduce their number while preserving navigability [20].

The Headland Generation step defines turning areas for agricultural machinery, either using a constant width [21] or minimizing headland width individually for each side to reduce wasted space [22]. Following this, Swath Generation covers the internal field area with either straight or curved paths. Straight swath generation involves selecting an optimal coverage angle, which can be identified through brute-force search [23], optimization methods [24], or constrained angle searches [25]. In fields with elevation changes, curved swaths may outperform straight swaths by saving energy and avoiding challenging coverage angles. Curved swaths can follow the longest curved field edge [26] or they are a compromise between the curves of opposing field edges [27].

Next, Route Planning sorts generated swaths efficiently, often employing metaheuristic optimization methods reviewed comprehensively by Filip et al. [17]. The final step, Path Planning, connects sorted swaths into navigable routes, either swath-to-swath directly [13] or via headlands [28], ensuring paths have smooth, drivable turns suitable for robotic vehicles.

The above steps are executed sequentially. However, some methods [29,30] aim to simultaneously solve more than one step of the ACPP problem, without following the sequential order. For example, to generate headlands (step 2) with the minimum width on each border [31], the coverage angle of the swaths has to be calculated (step 3) before handling step 2.

## 3. The Fields2Benchmark open-source benchmark

Fields2Benchmark organizes the evaluation of ACPP in two interconnected blocks: Field Arrangement and Route and Path Planning. The Field Arrangement block involves the Field Decomposition, the Headland Generator, and the Swath Generator modules. The Route and Path Planning block comprises the Route Planning and Path Planning modules. As shown in Fig. 2, the framework operates as follows. First, the benchmark requires the specification of input parameters, being physical parameters and use case decisions. Next, for each module within the framework, a set of selected algorithms is initialized. Simultaneously, objective functions are defined to assess the outputs of each module. The benchmark then executes the modules sequentially, where the outputs of a preceding module serve as the inputs for the subsequent module. Two cases break the sequential order. First, when an earlier stage needs data that a later module provides, the framework calls the latter module, retrieves the data, and then resumes the normal sequence. For instance, if the headland width needs to be dynamically calculated, the Swath Generator must first be run to determine the coverage angle. Second, when an earlier module's objective relies on outputs from a later one. For example, the Route Planning uses a Path Planning method to measure the path length of a route. The benchmark outputs are recorded in a database and saved as images to ease further processing.

### 3.1. Input parameters

A wide range of applications are supported by means of a configuration step to define the use case. The settings of this configuration are classified into physical parameters and use case decisions.

#### 3.1.1. Physical parameters

The physical parameters concern physical measures, such as the robot's dimensions and the geometry of the field to cover.

The robot is defined by its physical width, its coverage width –also called operational width–, and its minimum turning radius. For some operations, such as fertilization and harvesting, the robot has a capacity

[1] https://github.com/Fields2Cover/Fields2Cover.

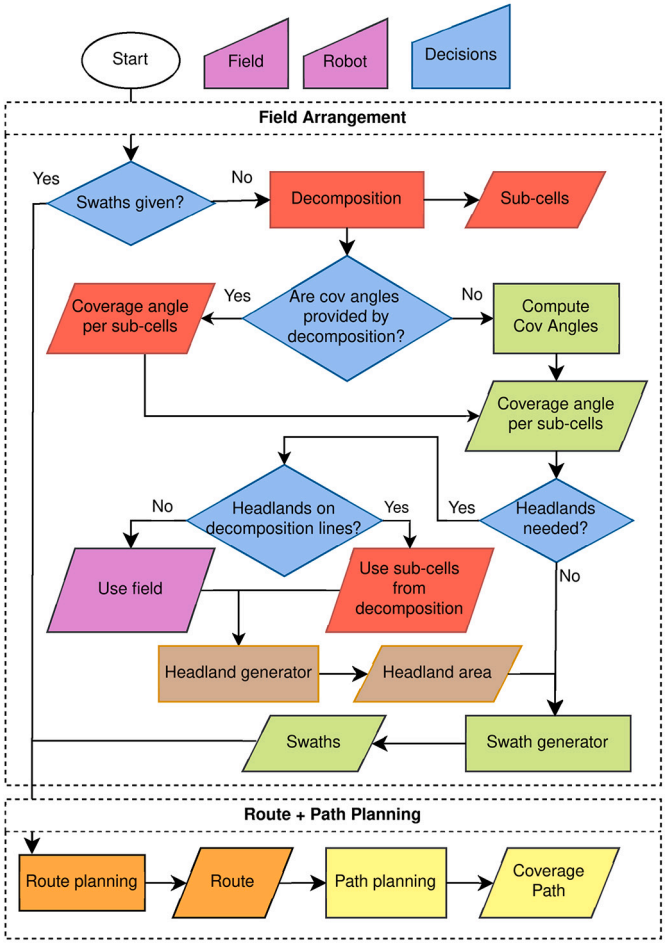



**Fig. 2.** Fields2Benchmark workflow covering all processing steps from input configuration to the final coverage path output. Colors denote different inputs and modules, where purple represents the physical inputs of the robot and the field; blue indicates use case decisions; red, green, and brown correspond to the Field Decomposition, Swath Generator and Headland Generator modules in the Field Arrangement block; orange and yellow denote the Route Planning and Path Planning modules, respectively.

that is emptied or filled during operation. The capacity is characterized by a rate, a maximum capacity, and a switch indicating whether it is used continuously (e.g., fuel consumption), or only during swath coverage (e.g., fertilization). Whenever planning a capacitated operation, the field representation also requires reloading points, which are sites used to refill or empty the robot during the operation.

The field boundaries are defined as polygons; the outer ring delineates the field's external boundary while the inner rings indicate obstacles [32]. The field geometry can be specified in any coordinate reference system, as Fields2Benchmark has functionality to transform global points to a local reference system. If the points are in geographic (lat. long.) coordinates, they are first converted to UTM and then shifted to a local system whose origin is at the first point of the field's outer border. Once a coverage path is computed, the results can be reverted to the original reference system. The field definition further allows specification of the start and end locations of field operations.

#### 3.1.2. Use case decisions

Apart from the specifications of machinery and field geometry, agricultural operations have specific needs, which add constraints to a general ACPP problem. For example, a lawnmower can arbitrarily cross the inner field to traverse between any two points, but a harvester would damage the standing crop when doing so. The constraints are expressed as use case decisions, which consist of:

- Are the swaths pre-established? In plantations such as orchards [33], the swaths have already been established so that the ACPP only concerns routing and path planning. In scenarios such as orchards where swaths are pre-established, predetermined swaths are utilized, thereby omitting the Field Arrangement block from the evaluation.
- Coverage angle given or computed? When swaths are not provided but a preferred direction is known, the coverage angle may be specified to generate the swaths accordingly. Alternatively, if no coverage angle is specified, the Swath Generator module employs optimization algorithms based on several objective functions to determine an optimal angle.
- Headland width options. There are four possibilities to choose this width: 1) half of the physical width of the robot, e.g. for holonomous vehicles; 2) the minimum width required for turns; 3) the product of the robot's coverage width and an integer, e.g. to have non-overlapping headland tracks; 4) no headlands, for cases like Unmanned Aerial Vehicles (UAV).
- Headland at decomposition lines. In order to mitigate in-field turning maneuvers, headlands are generated along the decomposition lines that segment a non-convex field into sub-cells; however, their generation is not always mandatory.

### 3.2. Field Arrangement block

The algorithms and objective functions implemented in the Field Arrangement block are summarized in Table 1.

#### 3.2.1. Decomposition module

Fields with concavities and obstacles may benefit from splitting into sub-cells, which are covered separately. Although the decomposition of fields into sub-cells can enhance coverage performance, it remains an optional step in the benchmark framework. The Decomposition module offers two algorithms: Trapezoidal cellular decomposition [34] and boustrophedon cellular decomposition [35]. The trapezoidal cellular decomposition uses each vertex point of the field to generate split lines. The boustrophedon cellular decomposition is similar to the trapezoidal decomposition, but only splits in specific vertex points. Both algorithms rely on a split angle to make the decomposition. The optimal split line is searched by evaluating all angles between 1 and 360 degrees with a step of 1 degree, which can be adapted.

The objective function provided in this module is the Minimum Sum of Altitude (MSA) [36]. The altitude is defined as the widest width of a polygon, which is proportional to the minimum number of swaths that covers it. The MSA returns the sum of the altitude of each sub-cell.

#### 3.2.2. Headland generator module

The Headland Generator module provides two algorithms: Constant width and required width. The constant width algorithm [26] inwardly buffers the field borders by a value that is common for all the borders. In contrast, the required width algorithm computes the offset by equation [31]:

$$H_i = r_{rob} * (sin(\theta - \gamma_i) + 1) + w_{rob}/2 \quad (1)$$

where $H_i$ is the headland width required to turn, $\theta$ is the coverage angle, $\gamma_i$ is the angle of the segment $i^{th}$ of the border, and $w_{rob}$ and $r_{rob}$ are the width and the minimum turning radius of the robot, respectively. The required width algorithm requires that the coverage angle be determined prior to the generation of the inner field; therefore, the angle must be either specified directly or computed via the Swath Generator module.

The headland width for the constant width algorithm is typically provided by the farmer. If not entered, an automatic criterion is used. As a means of precaution, this implementation picks the worst case scenario from Eq. (1):

**Table 1**
List of algorithms and cost functions available in the Field Arrangement block of Fields2Benchmark, for each of the different modules.

| | | Algorithms | Objective Functions |
|---|---|---|---|
| Field Arrangement | Decompositions | Any Field:<br>No decomposition (ND)<br>Non-Convex Fields:<br>Trapezoidal (TD)<br>Boustrophedon (BD) | Min-Sum-Altitude (MSA) |
| | Headland Generators | Constant Width (CW)<br>Required Width (RW) | Remaining mainland area (RMA) |
| | Swath Generators | Any Field:<br>Longest Edge (LE)<br>Brute Force (BF)<br>Oksanen search (OS)<br>Fields with Curved Borders:<br>Curved Swaths (CS) | Number of swaths (NS)<br>Sum of Swath Lengths (SL)<br>Field coverage (FC) |

$$H_i = r_{rob} * 2 + w_{rob}/2 \quad (2)$$

The distance between the first headland pass and the field borders is at least $w_{rob}/2$, whilst adjacent headland tracks are separated by the robot coverage width.

The objective function of the Headland Generator module is the relative inner-field area, which is the ratio between the inner-field area and the total area. The optimization function maximizes the relative inner-field area.

### 3.2.3. Swath generator module

Swaths are generated by offsetting a seed line at a distance equal to the coverage width of the robot until the field is completely covered. For a straight seed line, its angle — the coverage angle — determines the direction of all the swaths. As a rule of thumb, farmers commonly select the direction of the longest edge of the field [37]. The coverage angle can also be searched using optimization methods. Two of those optimizers are the brute force search over a discrete set of angles [23,31,21], and the Oksanen method [25,24], which first identifies a set of angles, to then proceed with a greedy search around the best angles of the first step. Both optimizers require an objective function to evaluate the results. The objective functions provided by Fields2Benchmark are:

- Number of swaths. In this work the Jin approximation [20] is used: $N_i = L_i \frac{|sin(\theta - \alpha_i)|}{2 * w_{rob}}$, where $N_i$ is the number of turns, $\theta$ is the coverage angle, $w_{rob}$ is the robot width, and $L_i$ and $\alpha_i$ are the length and the direction of the edge of the border, respectively. This function should be minimized.
- Sum of swath lengths. This function should be minimized.
- Field coverage, i.e., the ratio between the area covered by the swaths and the total area of the field. This is a maximization objective function.

When the borders of the field are curved segments, the swaths can be generated following a curved seed line [26]. Fields2Benchmark provides an algorithm to generate swaths following the longest curved edge. A curved edge is defined as a sequence of several straight segments in which the angle difference between the direction of two consecutive segments is smaller than a threshold. A parallel swath is generated at a distance $s$ from a seed curve by first constructing bisectors for each pair of consecutive segments. Subsequently, a point located at distance $s$ along each bisector is selected; the concatenation of these points yields the resulting curved swath.

## 3.3. Route and Path Planning block

The Fields2Benchmark algorithms and objective function of the Route and Path Planning block are listed in Table 2.

### 3.3.1. Route planning module

Generating a route given a set of swaths involves tasks: 1) sorting the coverage order of the swaths, and 2) connecting the swaths with turns.

In a convex field, the swaths can be sorted following a predefined pattern [38]. The patterns provided by Fields2Benchmark are: Boustrophedon, in which swaths are covered sequentially; Snake pattern, in which at each end point one swath is skipped, covering the field using the odd swaths first, followed by the even swaths in reversed order; Spiral pattern, which covers blocks of $n$ swaths each time from opposite sides of the field towards the center. Another possibility is optimizing the coverage by minimizing an objective function, which is referred to as B-patterns [39].

The patterns assume that adjacent swaths in the coverage order have opposite coverage directions, so that the end of one swath and the start of the next swath are at the same headland border. However, fields with concavities or obstacles do not follow that rule, requiring optimization of the order of the swaths and their final direction. To achieve this, Fields2Benchmark uses the or-tools library [40] to implement the B-patterns method. This method receives the costs of driving between swaths extremes and to the start and end point, and returns the coverage order of the swaths and their direction. Note that the B-patterns algorithm relies on automated hyperparameter tuning as implemented in the or-tools library.

Minimization of the driving costs requires all possible connections between pairs of swaths in the field. A graph is used to find the shortest path between the extremes of the swaths. The headlands tracks are also included in the graph. As the extremes of the inner field swaths are not placed directly on the headland tracks, the extremes of these swaths are connected by snapping to the closest point in the headland tracks. The shortest path to traverse the headlands from one point to the other is found by the Floyd-Warshall algorithm [41].

The B-patterns method has been extended to support capacitated robots. A robot capacity is considered full at the start of the route if it empties along the path, or empty if it fills. The capacity rate depends on its type; capacity may only concern operations on swaths, e.g. fertilizing, or continuous operation, e.g. fuel consumption. For each capacity, at least one reload point is required, which is a point at which the capacity returns to the state it was at the start of the route. More than one reload point can be set at the same location so as to serve multiple visits. Reload points are incorporated into the B-patterns method as optional nodes, and are connected to the nearest points on the headland tracks within the shortest path graph. Fields2Benchmark supports route planning for a robot with multiple capacities, and with one or more reload points for each capacity.

The route planning module provides two objective functions: the route length and the path length. The route length refers to the total distance of the planned route that connects all swaths within the field. The path length, on the other hand, is the total distance of the path gen-

**Table 2**
List of algorithms and cost functions available in the Route and Path Planning block of Fields2Benchmark, for each of the different modules.

| | | Algorithms | Objective Functions |
|---|---|---|---|
| Route and Path Planning | Route Planning | Convex Fields: | Route Length<br>Path Length |
| | | Boustrophedon | |
| | | Snake | |
| | | Spiral | |
| | | Any Field: | |
| | | B-Patterns w/o capacity | |
| | | B-Patterns with capacity | |
| | Path Planning | Dubins<br>Reeds-Shepp | Path Length |

erated by applying a path planning algorithm to the route. This includes detailed movements, such as turns and transitions between swaths, as computed by the path planning module. While the route length provides a higher-level measure, the path length captures the movement details. Both objective functions are minimized.

#### 3.3.2. Path planning module

Transforming a route into a complete path involves inserting smooth connections between swaths, which take the holonomic constraints of the robot into account. When the endpoints of two swaths are on the same headland border, the turn between them can be computed using either Dubins' curves [42] or Reeds-Shepp's curves [43]. Both algorithms generate the shortest path between two positions using straight and circular segments with the robot's minimum turning radius. The key difference is that Dubins' curves only allow forward movements, whereas Reeds-Shepp's curves also permit reverse movements. Both methods are computed using segments to find the best set of straight and circular movement. That best set is discretized to obtain a vector of points, which is the path for the turn.

For connections involving transitions between different headland borders, the problem becomes more complex. Unlike turns between nearby swath endpoints, navigating along headland tracks requires determining the start and end points of each turn along the sequence. Here, the center point in each set of three consecutive points along the sequence is considered a corner. The distance from the start and end points of the turn to the corner, $d_c$, is approximated as the minimum turning radius of the robot. Points at a distance $d_c$ from the corner are generated along the intersecting segments to serve as the start and end points of the turn. Dubins or Reeds-Shepp curves are then used to compute the shortest path between these points, ensuring that the turns respect the robot's motion constraints. If the required turning distance exceeds the length of a segment on either side of the corner, the corresponding point is omitted. In cases where the turning distance for two contiguous corners exceeds the shared segment length, the non-shared segments are extended until they intersect, thereby producing a new corner. This process is iteratively repeated to ensure feasible turns.

The objective function provided by Fields2Benchmark is the total path length, which quantifies the overall distance traveled by the robot, including all turning maneuvers.

### 3.4. Benchmark outputs

The results generated by Fields2Benchmark are recorded in a structured database. Since Fields2Benchmark supports multiple fields simultaneously, the outputs for each field are saved in separate folders. These folders contain all relevant data, including both numerical metrics and visual outputs.

The metrics, which are calculated by the previously defined objective functions are logged in a file named after the experiment. Each entry includes the following information:

- Algorithm identifiers for each module in the workflow.
- Values of the objective functions.
- Processing times for each step in the workflow and for evaluating each objective function (Tables 1 and 2).
- Metadata such as the field area, the number of points defining the outer border, and the decisions applied.

Moreover, images are generated for each module, providing a graphical representation of the module outputs. These images enable a visual comparison of results alongside numerical data.

### 3.5. Benchmark implementation

The benchmark is implemented in C++17 with an object-oriented approach. Each algorithm (see Tables 1 and 2) is encapsulated as a class that inherits from a module-specific base class. This base class defines methods such as initialization, execution, and result logging. This structure ensures consistency across the Field Arrangement and Route and Path Planning blocks.

Objective functions follow a similar design. A general base class captures the core concept of an objective function, and module-specific base classes extend this functionality. Each module pairs with objective functions to evaluate its performance. For every module, we maintain a collection of algorithm instances—each stored as a pointer to its base class—along with its associated objective functions.

This design, implemented from scratch based on the literature, enhances modularity and facilitates the integration of new methods.

## 4. Results

The results are organized along three use cases: 1) field arrangement; 2) route and path planning without capacity constraints; 3) route and path planning with capacity constraints. These use cases allow the analysis of each block separately, showing the capabilities of the benchmark. Each use case is demonstrated using a different field to showcase Fields2Benchmark's ability to handle complex geometries while keeping the number of figures in this work reasonable. The used fields are depicted in Fig. 3. These examples serve as illustrative cases rather than statistically representative samples of the 350 fields.

To further evaluate the performance of the ACPP algorithms provided with the benchmark, an experiment was carried out focusing on the processing time required for each method. These results are shown per module to provide some insights in their relationship with the inputs. This experiment was done on 350 fields, including the fields of the use cases.

Fields used for the experiments in this paper were compiled into a publicly-available dataset.[2] These fields were extracted and processed from the EuroCrops dataset [46], manually selecting fields from The Netherlands, Estonia, and Lithuania.

[2] Dataset can be found at: https://doi.org/10.5281/zenodo.14524735.

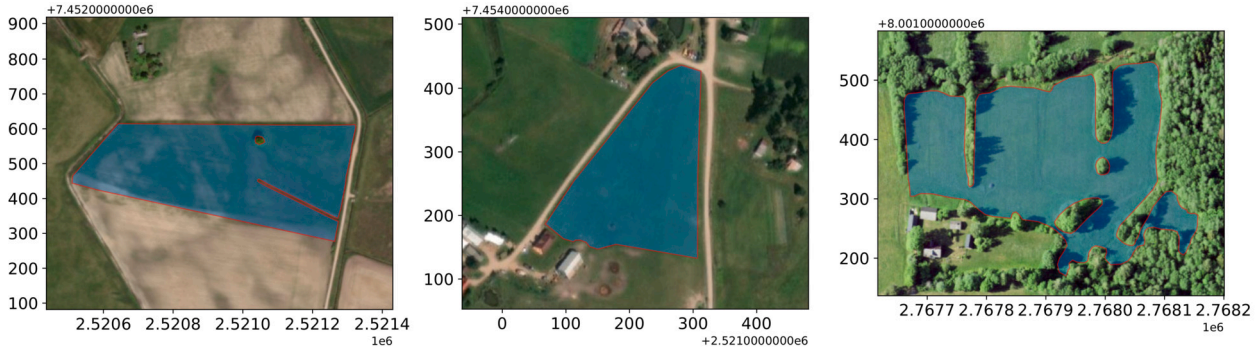



**Fig. 3.** Satellite images of the fields used in the use cases 1-3, from left to right respectively. The first example features a non-convex border with an obstacle, the second shows a triangular field with a curved border to demonstrate curved swath generation, and the third presents an extremely challenging case with multiple obstacles and narrow corridors to challenge the complete workflow, including capacity route planning. Imagery source: were created using ESRI imagery [44] available through the OpenStreetMap project [45].

**Table 3**
Selection of results from the benchmark of the Field Arrangement block of use case 1. Algorithms and objective functions complete names can be found in Table 1[1].

| Algorithm | MSA (m) | RMA | NS | SL (m) | FC |
|---|---|---|---|---|---|
| 1) ND + CW + LE | **202.8** | 0.867 | 79 | 1.76e4 | 0.867 |
| 2) ND + CW + CS | **202.8** | 0.867 | 79 | 1.76e4 | 0.867 |
| 3) ND + RW + LE | **202.8** | **0.900** | 79 | 1.83e4 | 0.900 |
| 4) ND + RW + (BF+NS) | **202.8** | **0.900** | 79 | 1.81e4 | 0.890 |
| 5) ND + RW + (OS+NS) | **202.8** | **0.900** | 79 | 1.83e4 | **0.901** |
| 6) BD + CW + (BF+NS) | 238.5 | 0.764 | 64 | 1.50e4 | 0.741 |
| 7) TD + CW + (BF+SL) | 241.3 | 0.480 | **36** | **8.23e3** | 0.406 |

[1] ND - No decomposition; BD - Boustrophedon decomposition; TD - Trapezoidal decomposition; CW - Constant width; RW - Required width; LE - Longest edge; CS - Curved swaths; BF - Brute force; OS - Oksanen search; MSA - Minimum sum altitude; RMA - Remaining mainland area; NS - Number of swaths; SL - Sum of swath lengths; FC - Field coverage.

The robot employed was the same for all the experiments, except for the limited capacity in the use case 3. This robot has a $3m$ coverage width, a $3m$ physical width, and a minimum turning radius of $2m$. The following use case decisions apply: 1) the swaths were generated by the Swath Generator module; 2) the headland width is the minimum required, and 3) headlands are created around the decomposition lines.

### 4.1. Use case 1: field arrangement

The first use case employed the benchmark to compare the modules of the Field Arrangement block. This use case employs the first field shown in Fig. 3. The field covers an area of 6.09 ha and contains a single circular obstacle. It was chosen because a fence disrupts its trapezoidal shape, and although the borders are nearly straight, they consist of many discrete points. This structure allows analysis of how the number of points along the border influences the field decomposition. All algorithms in Table 1 were provided to the benchmark, taking into account that both brute force and Oksanen search were optimized against the number of swaths and the sum of swath lengths. Accordingly, 36 combinations were tested in this use case. In Table 3, seven of these combinations are shown, which were selected because they provide the best values for the objective functions for this specific use case, or because they used an algorithm not used for the better ranked combinations.

Fig. 4 portrays the combinations 2, 5, 6 and 7 from the Table 3. For this particular field, curved swaths (Fig. 4a) do not provide any benefit in any of the objective functions used (Table 1, row 2). The headlands in Fig. 4a and Fig. 4b can be visually compared, as Fig. 4b uses the required width algorithm, requiring less headland area. The southern border of the field benefits the most from the selection of the Required width algorithm in Fig. 4b. Fig. 4c and Fig. 4d show the effects of the boustrophedon and trapezoidal decompositions, respectively. Combination 7 in Table 3 (Fig. 4d) optimizes the number of swaths and sum of swath lengths, but it provides the worst Remaining Mainland Area value. This is because the trapezoidal decomposition generates a greater number of sub-cells, each requiring a headland around it.

**Table 4**
Benchmark of the Route and Path Planning block for the use case 2.

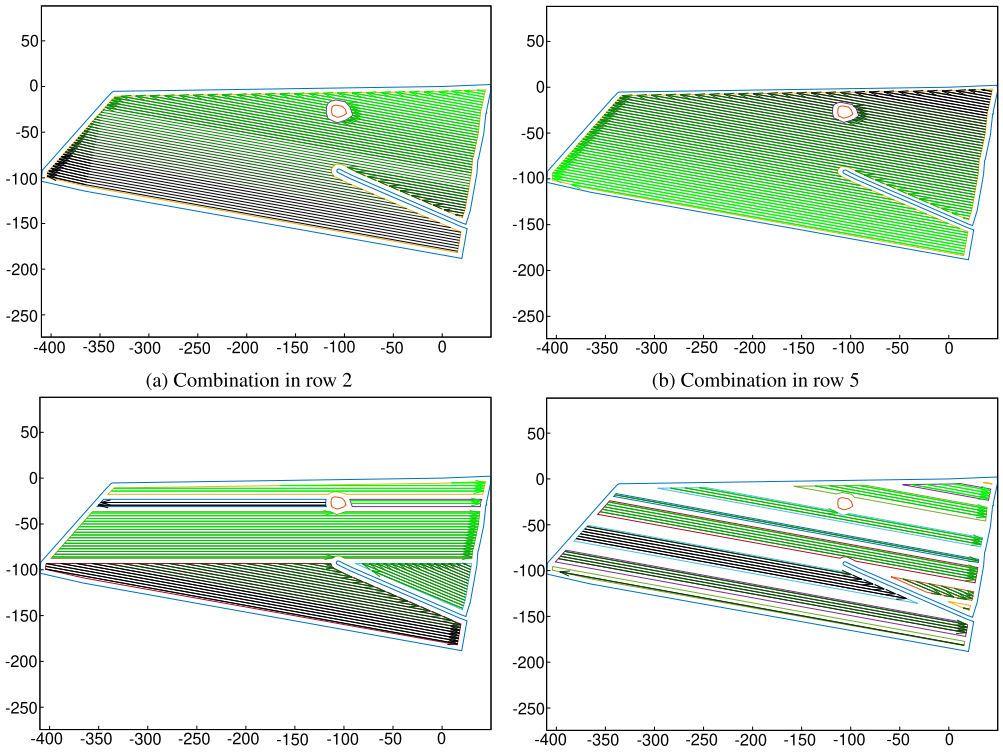
| Algorithm | Route Length (m) | Path Length (m) |
|---|---|---|
| 1) B-Patterns + Dubins | 4390.4 | 4336.2 |
| 2) B-Patterns + Reeds-Shepp | 4390.4 | 4211.2 |
| 3) Boustrophedon + Dubins | **4383.8** | 4336.7 |
| 4) Boustrophedon + Reeds-Shepp | **4383.8** | **4205.4** |
| 5) Snake + Dubins | 4548.3 | 4374.5 |
| 6) Snake + Reeds-Shepp | 4548.3 | 4327.5 |
| 7) Spiral + Dubins | 4660.4 | 4522.2 |
| 8) Spiral + Reeds-Shepp | 4660.4 | 4471.0 |

### 4.2. Use case 2: route and path planning without capacity constraints

The second use case plans the route and the path of a robot without capacities to test the Route and Path Planning block. In this case, the swaths are given as generated using the constant headland and the curved swath algorithm while skipping the decomposition step. The field selected for this use case is nearly convex and without obstacles, which allows using patterns for sorting the swaths. The compared algorithms for the route planning module are the B-patterns, the boustrophedon pattern, the snake pattern and the spiral pattern with $n = 6$ as block size, and for the path planning module, the Dubins' and the Reeds-Shepp's curves. All the combinations concerning this use case are shown in Table 4. Paths generated by the combinations 1, 4, 5 and 8 are illustrated in Fig. 5 to provide further insights.

The boustrophedon pattern had the shortest route length for this specific field (Table 4). This result makes sense as the parallel swaths are covered sequentially, which minimizes the distance traveled through the headlands. The B-patterns algorithm should also have discovered this pattern, but it encountered a local minimum during the optimization, which prevented finding the optimal solution. The routes of the snake and spiral patterns were longer than the boustrophedon and B-patterns.

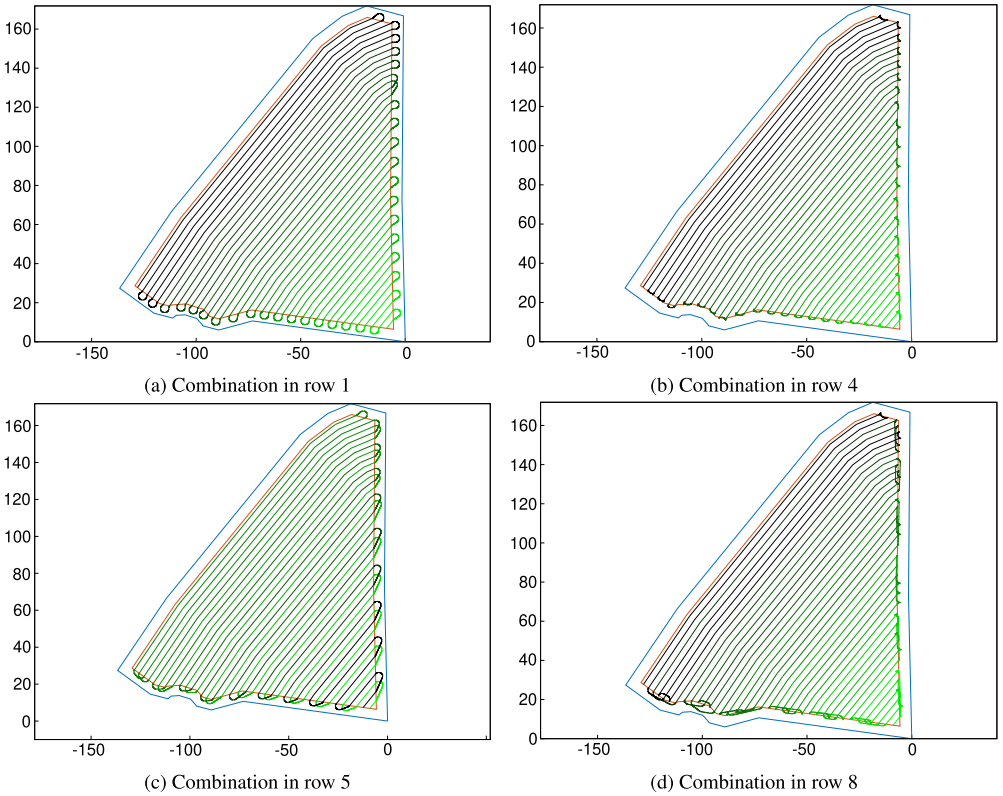
In all the combinations, Reeds-Shepp's curves outperformed Dubins', but this is because the turns are partly inside the inner field. Turning in the inner field may damage the crop, making it unsuitable for many agricultural operations. This finding reveals the relevance of providing visual results by the benchmark. Simply adapting the algorithm by offsetting the swaths by a fixed amount would avoid turns inside the inner field.




(a) Combination in row 2 (b) Combination in row 5

(c) Combination in row 6 (d) Combination in row 7

**Fig. 4.** Headland and swaths generated by a selection of the combinations of algorithms from Table 3. The XY-axis are plotted in meters, using as origin a point in the field border. The outer field border is shown in blue.




(a) Combination in row 1 (b) Combination in row 4

(c) Combination in row 5 (d) Combination in row 8

**Fig. 5.** Paths generated by a selection of the combinations of algorithms from Table 4. The XY-axis are plotted in meters, using as origin a point in the field border. The outer field border is shown in blue.

### 4.3. Use case 3: route and path planning with capacity constraints

The last use case aims to plan a coverage path for a capacitated robot in a complex field. The swaths were generated using the same algorithms as for the previous use case, and then connected using the B-patterns algorithm and Dubins' curves with and without capacity constraints.

The field used for this use case and the planned coverage paths are shown in Fig. 6. For this experiment, the start and end points of the route and the reload point are located at different points on the outer border of the field. With capacity constraints (Fig. 6b), the robot has a single capacity, which is only deployed on the swaths and allows covering $3000m$. Knowing that the total swath length is around $6302.2m$, the robot has to reload twice to complete the operation.

The headland paths shown in Fig. 6a and Fig. 6b approach the field boundary closer than half the width of the robot. This problem is caused by the smoother used to simplify the headland paths, which is applied without checking whether the final path is contained within the field.

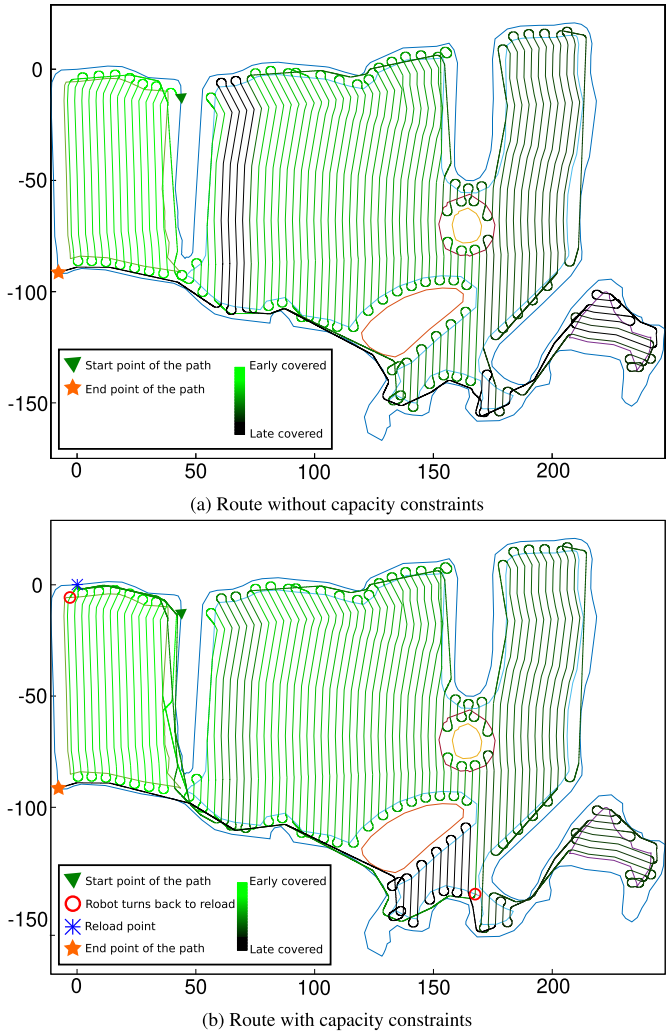
The optimized path lengths are $8359m$ and $8655m$ for the non-capacitated and capacitated problem, respectively. The difference between both path lengths is due to the cost of traveling to the reload points. The coverage order of the swaths is similar in both cases, differing from each other in the reloading paths when needed, and the coverage order of a sequence of four swaths. With capacity constraints, optimized route length is $8621m$. Contrary to the previous use case, the route length is shorter than the path length, because of the complexity of the headland passes. These results demonstrate the benchmark's capability of optimizing coverage paths for robots with limited capacity.

### 4.4. Processing time

In this last experiment, the computational performance of the benchmark was analyzed. For this purpose, 350 fields were tested across all five modules. Running the benchmark experiment with all fields took 12 hours, on an Intel(R) Xeon(R) E-2276M CPU.

The observed relationships between the processing time and the inputs for each module are shown in Fig. 7. Only the plots showing a clear trend are presented here:

- Decomposition module (Fig. 7.A): the processing time of the algorithm was linearly related with the number of points in the outer border of the field. The trapezoidal decomposition was slower than the boustrophedon decomposition because more sub-cells were generated. Both decompositions also allowed skipping the Decomposition module, which is shown as "No decomposition". Interestingly, the processing time of this module was not related to the area of the field. The maximum processing time of this module for a field was 49.5 seconds by the trapezoidal decomposition, and the mean processing time was of 2.24 seconds.
- Headland Generator module (Fig. 7.B): For this analysis, the Decomposition module was skipped. Similarly to the Decomposition module, the processing time of this headland generator related to the number of points in the outer border of the field. The constant width headland generator grew linearly, while the required width algorithm increased faster, owing to the need to consider each segment of the border independently. The maximum processing time of this module for a field was 0.1 seconds by the required width algorithm, and the mean processing time was 2.5 milliseconds.
- Swath Generator module (Fig. 7.C): For this analysis, the decomposition was skipped, and the headland was generated with constant width. The processing time of this module increased with the inner field area. The longest edge algorithm was the fastest method, as the coverage angle was computed using only the field border. Moreover, Oksanen's search was faster than brute force owing to fewer evaluations of the objective function and the number of swaths objective function being faster than summing swath lengths. The maximum processing time of this module for a field was 5.77 seconds by the brute force optimizing the sum of swath lengths, and the mean processing time was 0.20 seconds.
- Route Planning module (Fig. 7.D): For this analysis, the swaths were generated with the brute force optimizer, minimizing the number of swaths from the previous module. The route planning processing time increased with the number of swaths. All algorithms, except that of the B-patterns, had similar processing times. This similarity was because the algorithms spent most time on generating the shortest path connections between swaths. In contrast, the B-patterns took more time because it relied on an optimizer. The maximum processing time of this module was 2.28 minutes using the B-patterns and the mean processing time was 4.52 seconds.
- Path Planning module (Fig. 7.E): This analysis used all the routes from the previous test. A trend was found between the number of route points and the processing time of the Path Planning module. There was a small difference between the processing times for Dubins and Reeds-Shepp, being Dubins a bit slower. Although theoretically Dubins should compute faster than Reed–Shepp, the timing includes the full execution of the module's algorithms. Since the benchmark times complete implementations, not just that of the




(a) Route without capacity constraints

(b) Route with capacity constraints

**Fig. 6.** Coverage paths obtained in use case 3, using B-patterns Dubins' curves. These paths are (a) without and (b) with capacity constraints. In (b), the robot has to reload twice during the operation. The reload point is located at (0,0) and the red circles indicate where the robot interrupts the swath coverage to reload. In both cases, the start point is represented by a green triangle, and the end point with an orange star. The path is traversed from green to black.

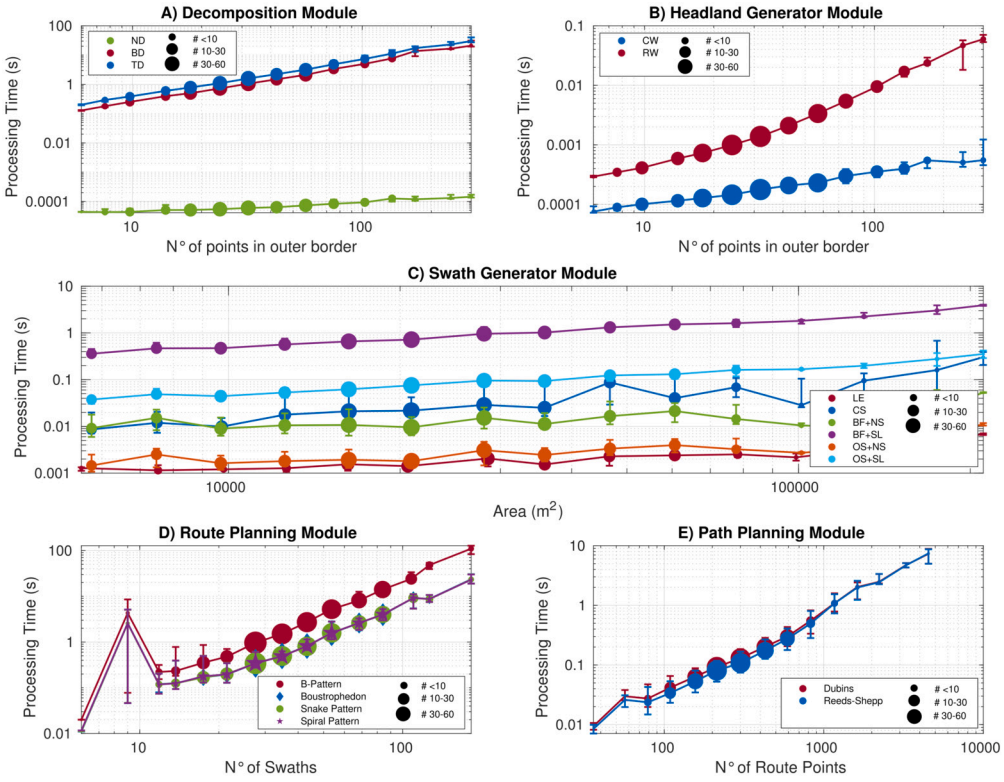



**Fig. 7.** Time analysis experiments. Data bins along the x-axis represent input sizes, while whiskers indicate the upper and lower quartile of processing times. Panels A - E correspond to the Decomposition, Headland Generation, Swath Generator, Route Planning, and Path Planning modules, respectively. Abbreviations in the panels A, B and C correspond to the notation used in Table 1.

core routines, Dubins' extra discretization step makes it slower than Reed–Shepp. The maximum processing time of this module for a field was 9.64 seconds using Dubins, and the mean processing time was 0.28 seconds.

## 5. Discussion

The Fields2Benchmark provides a structured framework to assess ACPP methods over diverse agricultural operations, complementing existing datasets such as those by Nilsson [11] and Pour [12]. It offers fields of varying complexity, including obstacles and non-convex shapes, overcoming issues of comparability, generalizability, and reproducibility identified in prior works [2].

Several unusual situations arose in our experiments that merit attention. In the first use case (Table 3, Fig. 4), the trapezoidal decomposition led to notably poor results, creating excessively small mainland areas despite optimizing for fewer swaths and shorter swath lengths (Fig. 4d). This outcome highlights an inherent trade-off in decomposition strategies previously noted but not explored deeply [31,17].

Another perhaps surprising result emerged in the second use case (Table 4, Fig. 5), where simpler heuristic approaches like boustrophedon surpassed more complex optimization algorithms under nearly convex conditions. Particularly, the B-patterns algorithm failed to achieve the aimed global optimum but rather stalled in a local minimum. Sorting the swaths to find a coverage route is a NP-hard problem, which means that the complexity time of the solvers grow exponentially with its size. Even with a small convex field, finding the global optimum using brute-force becomes a vastly time-consuming task [10]. Metaheuristic optimizers are not guaranteed to find the global optimum, and the reported result demonstrates the inherent complexity of the problem. This result aligns with findings from structured environment studies [38]. Other studies [47] have proposed algorithms for full and partial field coverage path planning that do not rely on metaheuristics. These approaches open a new direction for research.

In the third use case (Fig. 6), capacity constraints were introduced to compare how resource limitations affect coverage routes. When the robot must reload, the total path length increases. This behavior correlates with field complexity and resource layout. In Fields2Benchmark, the capability to handle curved swaths (Figs. 4a, 5 and 6) is shown for all the steps of the ACPP workflow, including the route planning with capacity constraints.

Processing time analyses across 350 fields revealed that route planning, especially when optimizing B-patterns, represents the primary computational bottleneck (Fig. 7.D). Although the planning remains fast enough for practical use in autonomous workflows [48], exhaustive angle searches and detailed connection computations currently may limit scalability to larger or more intricate fields.

Nonetheless, challenges and limitations remain. The current benchmark does not handle all ACPP variants, such as headland coverage or fields with undulating terrain. The assumption that soil compaction is proportional to path length may simplify reality [49], and future comparisons could incorporate soil properties to further explore methods that minimize the impact of machinery on soil. Moreover, the path planning algorithms presented in this work –Dubins and Reeds-Shepp curves– ignore steering-rate bounds. This was done for conciseness purposes and to focus on the workflow. Nevertheless, it is noted that Fields2Cover includes functionality for the continuous versions of the here demonstrated algorithms.

The benchmark currently employs a limited set of evaluation metrics, primarily focused on path length, number of swaths, and processing time. While these metrics provide valuable insights, they overlook important practical considerations such as execution safety, machine wear, and fuel consumption, which can have significant operational impacts. The modular design of Fields2Benchmark supports future expansion,

with the potential to incorporate additional metrics and features, including dynamic re-planning, multi-robot coordination, and energy consumption models, to further enhance its scope.

The proposed framework omits headland coverage. Although researchers commonly skip it in ACPP [25,29], headland coverage has gained recent interest. Pour et al. [50] handles headlands by performing turns with Dubins and Reeds–Shepp curves, while Mier et al. [51] plans turns with continuous curvature paths. In Fig. 6 some paths collides with the border of the field. This path cannot cover the field without the robot crossing its border. To solve that, [52] optimizes the headland path to avoid the headland borders while maintaining a smooth path. Other works [53,28] that solve the headland coverage problem are expected to be added in future releases of Fields2Cover. Integrating headland coverage into the ACPP benchmark remains future work.

In terms of computational scalability, the current approaches which employ exhaustive angle search may not scale efficiently to larger or more complex fields. In these cases, heuristic methods-such as choosing the direction of the field's longest edge [37]-can produce effective solutions even for those fields. Additionally, Fields2Benchmark functions as an offline planner and does not yet integrate real-time sensor feedback or adaptive re-planning, limiting its immediate applicability in dynamic environments where field conditions can change rapidly.

Furthermore, a wider support for other ACPP applications like orchard navigation [13] or UAV-based sensing [33] by using grid-based methods could further promote the adoption of the Fields2Benchmark. Although the current implementation relies on classical algorithms, this choice was intentional to provide a reliable baseline. The modular design of Fields2Benchmark permits the integration of new state-of-the-art methods. Novel techniques which are currently being explored are planned to be incorporated in future releases.

## 6. Conclusions

This work presents a standardized, open-source benchmark for ACPP, integrated with Fields2Cover to support the research community. The benchmark facilitates the comparison of algorithms across a wide range of conditions, including support for complex fields, capacitated robots, and use case decisions, like the use of predefined swaths, the selection of the coverage angle and different headland width options. By linking inputs, decisions, algorithms, and outputs, it helps researchers understand not just which methods perform best, but why.

Our benchmark further provides:

- **A general open-source ACPP benchmark tool**: Fields2Benchmark provides a documented codebase with ready-to-use tools for evaluating ACPP algorithms. It includes a dataset of 350 real-world fields, including non-convex fields and fields with obstacles. Results are recorded in database files and images to facilitate visual and statistical analysis.
- **Modular Framework**: The benchmark includes 16 algorithms and 8 objective functions, distributed across five independent modules, i.e., Field Decomposition, Headland Generation, Swath Generation, Route Planning, and Path Planning. Each module solves a single step of the ACPP problem, making its algorithms and objective functions interchangeable. This modularity enables easy integration of new algorithms and objective functions.
- **Support for use case customization**: This benchmark accommodates diverse operational constraints by accepting use case decisions to modify the general ACPP. It allows predefined or algorithmically generated swaths, adjustable coverage angles, and multiple options to determine the headland widths.
- **Integration with Fields2Cover**: To encourage community adoption, the framework of Fields2Benchmark extends the Fields2Cover library by adding support for non-convex fields and capacitated operations while maintaining the modular structure of Fields2Cover.[3]

By openly releasing the code and datasets, this benchmark is expected to become a core tool for consistent evaluation and improvement of ACPP algorithms, encouraging collaboration and inviting the research community to contribute to their further development.

## CRediT authorship contribution statement

**Gonzalo Mier:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Ana María Casado Faulí:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Data curation. **João Valente:** Writing – review & editing, Supervision, Conceptualization. **Sytze de Bruin:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

## Funding

This publication is part of the project "Fields2Cover: Robust and efficient coverage paths for autonomous agricultural vehicles" (with project number ENPPS.LIFT.019.019 of the research programme Science PPP Fund for the top sectors which is (partly) financed by the Dutch Research Council (NWO).

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Dataset can be found at: https://doi.org/10.5281/zenodo.14524735

## References

[1] S. Chakraborty, D. Elangovan, P.L. Govindarajan, M.F. ELnaggar, M.M. Alrashed, S. Kamel, A comprehensive review of path planning for agricultural ground robots, Sustainability 14 (15) (2022) 9156, https://doi.org/10.3390/su14159156.

[2] A. Utamima, A. Djunaidy, Agricultural routing planning: a narrative review of literature, Proc. Comput. Sci. 197 (2022) 693–700, https://doi.org/10.1016/j.procs.2021.12.190.

[3] Y. Li, Z. Guo, F. Shuang, M. Zhang, X. Li, Key technologies of machine vision for weeding robots: a review and benchmark, Comput. Electron. Agric. 196 (2022) 106880, https://doi.org/10.1016/j.compag.2022.106880.

[4] J.v. Kistowski, J.A. Arnold, K. Huppler, K.-D. Lange, J.L. Henning, P. Cao, How to build a benchmark, in: Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering, 2015, pp. 333–336.

[5] S. Matloob, P.P. Datta, O.P. Kreidl, A. Dutta, S. Roy, L. Bölöni, Waterberry farms: a novel benchmark for informative path planning, preprint, arXiv:2305.06243, 2023, https://doi.org/10.48550/arXiv.2305.06243.

[6] G. Mier, J. Valente, S. de Bruin, Fields2cover: an open-source coverage path planning library for unmanned agricultural vehicles, IEEE Robot. Autom. Lett. 8 (4) (2023) 2166–2172, https://doi.org/10.1109/LRA.2023.3248439.

[7] E.K. Raptis, K. Englezos, O. Kypris, M. Krestenitis, A.C. Kapoutsis, K. Ioannidis, S. Vrochidis, E.B. Kosmatopoulos, Cofly: an automated, ai-based open-source platform for uav precision agriculture applications, SoftwareX 23 (2023) 101414, https://doi.org/10.1016/j.softx.2023.101414.

[8] T.M. Driscoll, Complete coverage path planning in an agricultural environment, Ph.D. thesis, Iowa State University, Department of Computer Science, 2011.

[9] J.I.V. Gomez, M.M. Melchor, J.C.H. Lozada, Optimal coverage path planning based on the rotating calipers algorithm, in: 2017 International Conference on Mechatronics, Electronics and Automotive Engineering (ICMEAE), IEEE, 2017, pp. 140–144.

[3] https://github.com/Fields2Cover/Fields2Cover.

[10] E. Khosravani Moghadam, M. Vahdanjoo, A.L. Jensen, M. Sharifi, C.A.G. Sørensen, An arable field for benchmarking of metaheuristic algorithms for capacitated coverage path planning problems, Agronomy 10 (10) (2020) 1454, https://doi.org/10.3390/agronomy10101454.
[11] R.S. Nilsson, K. Zhou, Method and bench-marking framework for coverage path planning in arable farming, Biosyst. Eng. 198 (2020) 248–265, https://doi.org/10.1016/j.biosystemseng.2020.08.007.
[12] D. Pour Arab, C. Essert, Agricultural fields 2d and 3d models dataset, https://doi.org/10.5281/zenodo.10949632, 2024.
[13] L. Guevara, M.M. Michałek, F.A. Cheein, Headland turning algorithmization for autonomous n-trailer vehicles in agricultural scenarios, Comput. Electron. Agric. 175 (2020) 105541, https://doi.org/10.1016/j.compag.2020.105541.
[14] D. Bochtis, C. Sørensen, P. Busato, I. Hameed, E. Rodias, O. Green, G. Papadakis, Tramline establishment in controlled traffic farming based on operational machinery cost, Biosyst. Eng. 107 (3) (2010) 221–231, https://doi.org/10.1016/j.biosystemseng.2010.08.004.
[15] R. Linker, T. Blass, Path-planning algorithm for vehicles operating in orchards, Biosyst. Eng. 101 (2) (2008) 152–160, https://doi.org/10.1016/j.biosystemseng.2008.06.002.
[16] M.F. Jensen, D. Bochtis, C.G. Sørensen, Coverage planning for capacitated field operations, part ii: optimisation, Biosyst. Eng. 139 (2015) 149–164, https://doi.org/10.1016/j.biosystemseng.2015.07.002.
[17] M. Filip, T. Zoubek, R. Bumbalek, P. Cerny, C.E. Batista, P. Olsan, P. Bartos, P. Kriz, M. Xiao, A. Dolan, et al., Advanced computational methods for agriculture machinery movement optimization with applications in sugarcane production, Agriculture 10 (10) (2020) 434, https://doi.org/10.3390/agriculture10100434.
[18] D. Bochtis, T. Oksanen, Combined coverage path planning for field operations, in: Proc. Joint International Agricultural Conference, JIAC, 2009, pp. 521–527.
[19] L.D. Nielsen, I. Sung, P. Nielsen, Convex decomposition for a coverage path planning for autonomous vehicles: interior extension of edges, Sensors 19 (19) (2019) 4165, https://doi.org/10.3390/s19194165.
[20] J. Jin, L. Tang, Optimal coverage path planning for arable farming on 2d surfaces, Trans. ASABE 53 (1) (2010) 283–295, https://doi.org/10.13031/2013.29488.
[21] S. de Bruin, P. Lerink, A. Klompe, T. van der Wal, S. Heijting, Spatial optimisation of cropped swaths and field margins using gis, Comput. Electron. Agric. 68 (2) (2009) 185–190, https://doi.org/10.1016/j.compag.2009.06.001.
[22] M. Spekken, J.P. Molin, T.L. Romanelli, Cost of boundary manoeuvres in sugarcane production, Biosyst. Eng. 129 (2015) 112–126, https://doi.org/10.1016/j.biosystemseng.2014.09.007.
[23] I. Hameed, D. Bochtis, C.G. Sørensen, A.L. Jensen, R. Larsen, Optimized driving direction based on a three-dimensional field representation, Comput. Electron. Agric. 91 (2013) 145–153, https://doi.org/10.1016/j.compag.2012.12.009.
[24] T. Oksanen, et al., Path Planning Algorithms for Agricultural Field Machines, Helsinki University of Technology, 2007.
[25] T. Oksanen, A. Visala, Coverage path planning algorithms for agricultural field machines, J. Field Robot. 26 (8) (2009) 651–668, https://doi.org/10.1002/rob.20300.
[26] I. Hameed, D. Bochtis, C.G. Sørensen, M. Nøremark, Automated generation of guidance lines for operational field planning, Biosyst. Eng. 107 (4) (2010) 294–306, https://doi.org/10.1016/j.biosystemseng.2010.09.001.
[27] M. Spekken, S. de Bruin, J.P. Molin, G. Sparovek, Planning machine paths and row crop patterns on steep surfaces to minimize soil erosion, Comput. Electron. Agric. 124 (2016) 194–210, https://doi.org/10.1016/j.compag.2016.03.013.
[28] M. Höffmann, S. Patel, C. Büskens, Weight-optimized nurbs curves: headland paths for nonholonomic field robots, in: 2022 8th International Conference on Automation, Robotics and Applications (ICARA), IEEE, 2022, pp. 81–85.
[29] M. Höffmann, S. Patel, C. Büskens, Optimal guidance track generation for precision agriculture: a review of coverage path planning techniques, J. Field Robot. (2023), https://doi.org/10.1002/rob.22286.
[30] G. Mier, S. Vélez, J. Valente, S. de Bruin, Soil2cover: coverage path planning minimizing soil compaction for sustainable agriculture, Precis. Agric. 26 (4) (2025) 1–21, https://doi.org/10.1007/s11119-025-10250-4.
[31] M. Spekken, S. de Bruin, Optimized routing on agricultural fields by minimizing maneuvering and servicing time, Precis. Agric. 14 (2013) 224–244, https://doi.org/10.1007/s11119-012-9290-5.
[32] C. Kottman, Ogc Simple Features, 2002.
[33] S. Vélez, G. Mier, M. Ariza-Sentís, J. Valente, Integrated framework for multipurpose uav path planning in hedgerow systems considering the biophysical environment, Crop Prot. (2024) 106992, https://doi.org/10.1016/j.cropro.2024.106992.
[34] J.-C. Latombe, Robot Motion Planning, Springer Science & Business Media, 1991.
[35] H. Choset, Coverage of known spaces: the boustrophedon cellular decomposition, Auton. Robots 9 (3) (2000) 247–253, https://doi.org/10.1023/A:1008958800904.
[36] W.H. Huang, Optimal line-sweep-based decompositions for coverage algorithms, in: Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164), vol. 1, IEEE, 2001, pp. 27–32.
[37] D. Bochtis, S. Vougioukas, Minimising the non-working distance travelled by machines operating in a headland field pattern, Biosyst. Eng. 101 (1) (2008) 1–12, https://doi.org/10.1016/j.biosystemseng.2008.06.008.
[38] K. Zhou, A.L. Jensen, D.D. Bochtis, C.G. Sørensen, Quantifying the benefits of alternative fieldwork patterns in a potato cultivation system, Comput. Electron. Agric. 119 (2015) 228–240, https://doi.org/10.1016/j.compag.2015.10.012.
[39] D.D. Bochtis, C.G. Sørensen, P. Busato, R. Berruto, Benefits from optimal route planning based on b-patterns, Biosyst. Eng. 115 (4) (2013) 389–395, https://doi.org/10.1016/j.biosystemseng.2013.04.006.
[40] L. Perron, Operations research and constraint programming at Google, in: Principles and Practice of Constraint Programming–CP 2011: 17th International Conference, CP 2011, Perugia, Italy, September 12-16, 2011. Proceedings 17, Springer, 2011, p. 2.
[41] R.W. Floyd, Algorithm 97: shortest path, Commun. ACM 5 (6) (1962) 345, https://doi.org/10.1145/367766.368168.
[42] L.E. Dubins, On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents, Am. J. Math. 79 (3) (1957) 497–516, https://doi.org/10.2307/2372560.
[43] J. Reeds, L. Shepp, Optimal paths for a car that goes both forwards and backwards, Pac. J. Math. 145 (2) (1990) 367–393, https://doi.org/10.2140/pjm.1990.145.367.
[44] ESRI, Esri imagery, https://www.esri.com/en-us/home, 2025.
[45] OpenStreetMap contributors, Openstreetmap, https://www.openstreetmap.org, 2025.
[46] M. Schneider, A. Chan, M. Körner, EuroCrops, Zenodo (2023), https://doi.org/10.5281/zenodo.10118572.
[47] M.G. Plessen, Optimal in-field routing for full and partial field coverage with arbitrary non-convex fields and multiple obstacle areas, Biosyst. Eng. 186 (2019) 234–245, https://doi.org/10.1016/j.biosystemseng.2019.08.001.
[48] D.C. Slaughter, D. Giles, D. Downey, Autonomous robotic weed control systems: a review, Comput. Electron. Agric. 61 (1) (2008) 63–78, https://doi.org/10.1016/j.compag.2007.05.008, https://doi.org/10.1016/j.compag.2007.05.008.
[49] G. Mier, J. Valente, S. de Bruin, Optimizing agricultural coverage path to minimize soil compaction, in: Precision Agriculture'23, Wageningen Academic Publishers, 2023, pp. 467–472.
[50] D. Pour Arab, M. Spisser, C. Essert, Complete coverage path planning for wheeled agricultural robots, J. Field Robot. (2022), https://doi.org/10.1002/rob.22187.
[51] G. Mier, R. Fennema, J. Valente, S. de Bruin, Continuous curvature path planning for headland coverage with agricultural robots, J. Field Robot. (2024), https://doi.org/10.1002/rob.22489.
[52] M. Plessen, Smoothing of headland path edges and headland-to-mainfield lane transitions based on a spatial domain transformation and linear programming, preprint, arXiv:2407.05979, 2024, https://doi.org/10.48550/arXiv.2407.05979.
[53] M.G. Plessen, Freeform path fitting for the minimisation of the number of transitions between headland path and interior lanes within agricultural fields, Artif. Intell. Agric. 5 (2021) 233–239, https://doi.org/10.1016/j.aiia.2021.10.004.